

152113017: NESNE TABANLI PROGRAMLAMA I
FINAL SINAVI (CEVAPLAR)

Kitap ve notlar kapalı

Toplam: 100 puan, 3 Soru, Süre: 90 dakika

S.1	S.2	S.3	Toplam
20	26	54	100

Adı-Soyadı: CEVAPLAR

Numara: CEVAPLAR

1 ve 2nci soruları aşağıdaki programa göre cevaplayınız.

```
#include <iostream>
#include <string>
#include <list>
using namespace std;

class Konum {
private:
    int x;
    int y;
    float yon;
public:
    Konum(int _x=0, int _y=0, float
_yon=0)
        :x(_x), y(_y), yon(_yon) {}
    void setXY(int _x, int _y);
    void getXY(int &_x, int &_y) const;
    void setTh(float _yon);
    float getTh() const;
};

class Sekil {
private:
    Konum konum;
    const string tip;
public:
    Sekil(string _tip = "", Konum _konum
= Konum(0, 0)):tip(_tip), konum(_konum) {}
    void move(int x, int y);
    void rotate(float _yon);
    string getType() const;
    void print() const;
    void draw() const;
};

void Konum::setXY(int _x, int _y) {
    x = _x;
    y = _y;
}

void Konum::getXY(int &_x, int &_y) const {
    _x = x; _y = y;
}

void Konum::setTh(float _yon) {
    yon = _yon;
}

float Konum::getTh() const {
    return yon;
}

void Sekil::move(int x, int y) {
    konum.setXY(x, y);
}

void Sekil::rotate(float _yon) {
    konum.setTh(_yon);
}

string Sekil::getType() const {
    return tip;
}

void Sekil::print() const {
    int x, y;
    konum.getXY(x, y);
    cout << "Konum: (" << x << ", "
        << y << ", " << konum.getTh()
        << "), " << "Tip: " << tip;
}

void Sekil::draw() const {
    cout << "Sekil gosteriliyor." <<
endl;
}
```

S.1 (Templates, Total 20 points)

(a) [10 points] Konum sınıfını iki parametre alan (T1,T2) bir template sınıf olarak tekrar yazınız. Öyle ki, (x,y) çifti ve yon veri üyeleri farklı tipte olabilecektir (int, double, float, etc.). Örneğin, x ve y (bu iki üye aynı tipte, T1) üyeleri int, double, float tiplerinde olabileceği gibi, yon (bu üye x ve y den farklı olabilir, T2) veri üyesi de double, int, float olabilecektir. { İpucu: template sınıf ve fonksiyonların başında kullanılan şablon” template <typename T1, typename T2>” }

Cevap:

```
template<typename T1, typename T2>
class Konum {
private:
    T1 x;
    T1 y;
    T2 yon;
public:
    Konum(T1 _x = 0, T1 _y = 0, T2 _yon=0)
        :x(_x), y(_y), yon(_yon) {}
    void setXY(T1 _x, T1 _y);
    void getXY(T1 &_x, T1 &_y) const;
    void setTh(T2 _yon);
    T2 getTh() const;
};
```

```
template<typename T1, typename T2>
void Konum<T1,T2>::setXY(T1 _x, T1 _y) {
    x = _x; y = _y; }

template<typename T1, typename T2>
void Konum<T1,T2>::getXY(T1 &_x, T1 &_y)
const { _x = x; _y = y; }

template<typename T1, typename T2>
T2 Konum<T1,T2>::getTh() const {
    return yon; }

template<typename T1, typename T2>
void Konum<T1,T2>::setTh(T2 _yon) {
    yon = _yon; }
```

(b) [10 points] Şimdi, Konum sınıfı bir template sınıfı oldu. Bu durumda, Sekil sınıfında değişiklik yapma ihtiyacı var mıdır? Cevabınız evet ise, yapılacak değişiklikleri gösteriniz. Cevap hayır ise, nedenini açıklayınız? {İpucu: Örneğin, A, tek parametreye sahip bir template sınıf olsun. Bu durumda, parametresi int olacak bir A tipinde nesne yaratmak istersek, "A<int> aNesne;" şeklinde parametrenin ne olacağını belirterek nesneyi oluştururuz.}

Cevap:

```
class Sekil {
private:
    Konum<int,float> konum;
    const string tip;
public:
    Sekil(string _tip = "", Konum<int,float> _konum = Konum<int,float>(0, 0))
        :tip(_tip), konum(_konum) {}
    void move(int x, int y);
    void rotate(float _yon);
    string getType() const;
    void print() const;
    void draw() const;
};
```

S.2 (Operator Overloading & Exception Handling, Total 26points)

(i) [8 points] Operator (+=) fonksiyonunu global fonksiyon olarak yazınız. Öyle ki, operatörün sağındaki değer soldaki Konum nesnesinin yon üye değişkenine eklenecektir. (Örneğin, Konum kon(1,2,1.0); kon+=30.5; kodları çalıştıktan sonra, kon nesnesinin x=1, y=2, yon=31.5 olacaktır).

Cevap:

```
Konum operator+=(Konum &kon, float i){
    Kon.yon+=i;
    return kon;
}
```

(ii) [9 points] Konum sınıfının kurucu (constructor) fonksiyonunda, istisnai durumun (exception handling) kullanılması istenmektedir. Bu fonksiyonda, yon üyesine -180'den küçük veya 180'den büyük bir değer ataması yapılmak istenirse, istisna fırlatma (throw) yapılması istenmektedir. Fırlatılan değer string tipinde "out of range" değeri olacaktır. Buna uygun olarak, Konum sınıfının kurucu fonksiyonunu aşağıya yazınız.

Cevap:

```
Konum(T1 _x = 0, T1 _y = 0, T2 _yon=0) :x(_x), y(_y){
    if(_yon<-180 && _yon>180)
        throw (string) "out of range";
    yon=_yon;
}
```

(iii) [9 points] Aşağıdaki programı, (ii) şıkında yaptığınız değişiklik sonrası istisnanın yakalanması ve fırlatılan değerın ekrana yazdırılması için tamamlayınız.

Cevap:

```
int main(){

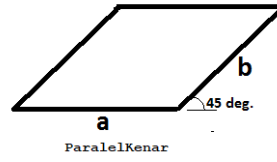
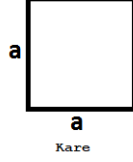
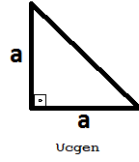
    try{

        Konum konumum(1,5, -200);

    }
    catch(string str){
        cout<<str<<endl;
    }

    return 0;
}
```

S.3 (inheritance&polymorphism, Total 54 points) Soruda aşağıdaki şekilleri dikkate alalım.



(i) [10 points] Yukarıdaki şekillerden Kare ve ParalelKenar sınıf kodlarını Sekil sınıfından miras alacak şekilde yazınız.

- Konum ve kenar değerlerini parametre olarak alan ve varsayılan değer olarak konuma $Konum(0, 0, 0)$, kenarlara 1 değer atayan bir varsayılan kurucu fonksiyon olacak.
- print ve draw fonksiyonlarını yazınız. Print fonksiyonu, üyeleri ve Sekilden miras aldığı üyeleri ekrana yazdıracak. draw fonksiyonu çizme işleminin yapıldığını belirten bir metni ekrana yazdıracak. Örneğin, "Ucgen".

Cevap:

```
class Kare:public Sekil {
private:
    int a;
public:
    Kare(Konum kon=Konum(0,0,0)
        ,int kenar=0)
        :Sekil("Kare")
        ,a(kenar) {}

    void print() const{
        Sekil::print();
        cout<<" , Kenar:"
            <<a<<endl;
    }

    void draw() const{
        cout<<"Kare" <<endl;
    }
};
```

Tangram
- sekilller: list<Sekil *> - iter: list<Sekil *>::iterator + add(Sekil *s): void + drawShape(): void + rotate(tip: string, yon: float) : void

```
class ParalelKenar:public Sekil {
private:
    int a;
    int b;
public:
    ParalelKenar(Konum kon=Konum(0,0,0)
        ,int kenar1=0, kenar2=0)
        :Sekil("ParalelKenar")
        ,a(kenar1),b(kenar2) {}

    void print() const{
        Sekil::print();
        cout<<" , Kenar:"
            <<a<<" , "<<b<<endl;
    }

    void draw() const{
        cout<<"ParalelKenar" <<endl;
    }
};
```

(ii) [8 points] Sekil sınıfını, soyut (abstract) taban sınıf yapmak istiyoruz. Bu amaçla, Sekil sınıfında yapılması gereken değişiklikleri gösteriniz. Sekil sınıfından miras ile türetilen her sınıfta mutlaka draw fonksiyonu kodlanması isteniyor. Print fonksiyonu isteğe bağlıdır. Cevabınızı buna uygun olarak verin. Değişiklik yapılan kodlamaları yazınız.

Cevap:

```
class Sekil{
    virtual void print() const{
        cout<<"Center: ("<<center.getX()<<" , "<<center.getY()<<" , "<<angle<<" , "
            "Color: "<<color<<" , "
            "Type: "<<type;
    }
    virtual void draw() const=0;
};
```

(iii) [18 points] UML Sınıf diyagramı yanda verilen bir Tangram sınıfı kodlayınız.

add() : Verilen Sekil (Kare, Ucgen, ParalelKenar) nesnesini listeye ekler.

drawShapes() : listedeki Sekil nesnelerinin birer birer print fonksiyonlarını çağırır.

rotate() : tip paramtresinde tip değerine sahip Sekil nesnesini listede bulup, yon değerini verilen değer olarak atar.

Tangram nesnesi yok edildiğinde, listesinde bulunan nesneler de yok edilecektir.

Cevap:

```
class Tangram {
private:
    list<Sekil *> sekilller;
    list<Sekil *>::iterator iter;
public:
    void add(Sekil *s);
    void drawShape();
    void rotate(string tip, float yon);
    ~Tangram();
};

void Tangram::add(Sekil *s) {
    sekilller.push_back(s);
}

void Tangram::drawShape() {
    for (iter = sekilller.begin(); iter !=
```

```
void Tangram::rotate(string tip, float yon) {
    for (iter = sekilller.begin(); iter !=
        sekilller.end(); iter++) {
        if (tip == (*iter)->getType())
            break;
    }
    (*iter)->rotate(yon);
}

Tangram::~~Tangram() {
    for (iter = sekilller.begin(); iter !=
        sekilller.end(); iter++)
        delete (*iter);
    sekilller.clear();
}
```

```

sekilller.end(); iter++)
    (*iter)->print();
}

```

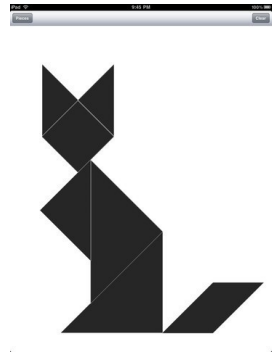
(iv) [8 points] Tangram nesnesini test etmek için bir program yazınız. Tangram nesnesi yandaki resimde bulunan, şekilleri nesne olarak listesinde bulunduracaktır. (3 küçük Ucgen, 2 büyük Ucgen, 1 Kare, 1 ParalelKenar) Sekil nesnelerinin Konum değerleri ve kenar değerlerini istediğiniz değer olarak atayabilirsiniz.

Cevap:

```

int main() {
    Tangram *tangram = new Tangram();
    //three small triangle
    tangram->add(new Ucgen(Point(1, 1), 0, 3));
    tangram->add(new Ucgen(Point(1, 1), 0, 3));
    tangram->add(new Ucgen(Point(1, 1), 0, 3));
    //two big triangle
    tangram->add(new Ucgen(Point(1, 1), 0, 3));
    tangram->add(new Ucgen(Point(1, 1), 0, 3));
    //one square
    tangram->add(new Kare(Point(5, 5), 30, 5));
    //one parallelogram
    tangram->add(new ParalelKenar(Point(6, 6), -60, 7, 3));
    cout << "----- Test 1 -----" << endl;
    tangram->drawShape();
    cout << endl << "----- Test 2 -----" << endl;
    tangram->rotate("Kare", 180);
    tangram->drawShape();
    delete tangram;
    return 0;
}

```



(v) [10 points] UML sınıf diyagramını çiziniz. Tasarımda kullanılan tüm sınıfları ve ilişkilerini (inheritance, aggregation, composition) göstermelisiniz. Sınıfları gösterirken, veri ve fonksiyon üyelerini göstermenize gerek yok.

Cevap:

